

Tutorial Passo a Passo

Documentação Swagger/OpenAPI

Projeto RabbitMQ - Envio de E-mails

Spring Boot 4.1.0 | SpringDoc OpenAPI 2.8.6 | Java 17

Índice

1. Introdução
2. Dependências Maven
3. Configuração do application.yaml
4. Classe de Configuração Swagger (SwaggerConfig)
5. Anotação @Tag no Controller
6. Anotação @Operation nos Endpoints
7. Anotação @ApiResponses e @ApiResponse
8. Anotação @Schema em DTOs e Models
9. @Content e @Schema em Responses
10. Como acessar o Swagger UI
11. Como baixar a especificação OpenAPI (JSON)
12. Tratamento de Erros com @RestControllerAdvice
13. Integração ErroResposta no Swagger
14. Como importar no Postman
15. Conclusão

1. Introdução

Este tutorial documenta o passo a passo para implementar e configurar a documentação Swagger/OpenAPI em uma API REST desenvolvida com Spring Boot 4.1.0 que utiliza RabbitMQ para envio de mensagens de e-mail.

A biblioteca utilizada é o SpringDoc OpenAPI (springdoc-openapi-starter-webmvc-ui), que gera automaticamente a especificação OpenAPI 3.0 e disponibiliza uma interface interativa (Swagger UI) para testar os endpoints.

Pré-requisitos

- Java 17+
- Spring Boot 4.1.0
- Maven
- Docker Desktop

2. Dependências Maven

No arquivo pom.xml, adicione a seguinte dependência dentro da seção <dependencies>:

```
<!-- SpringDoc OpenAPI (Swagger) -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>${springdoc-openapi.version}</version>
</dependency>
```

E no <properties>:

```
<springdoc-openapi.version>2.8.6</springdoc-openapi.version>
```

Essa dependência expõe automaticamente:

- /swagger-ui.html → Interface gráfica interativa
- /api-docs → Especificação OpenAPI em JSON

3. Configuração do application.yaml

No arquivo src/main/resources/application.yaml, configure os caminhos do Swagger:

```
springdoc:  
  api-docs:  
    path: /api-docs  
  swagger-ui:  
    path: /swagger-ui.html  
    operations-sorter: method  
    tags-sorter: alpha  
    try-it-out-enabled: true
```

- Propriedades:
- api-docs.path → Define a URL da especificação JSON
- swagger-ui.path → Define a URL da interface gráfica
- operations-sorter: method → Ordena endpoints por método HTTP (GET, POST...)
- tags-sorter: alpha → Ordena tags alfabeticamente
- try-it-out-enabled: true → Habilita o botão "Try it out" no Swagger UI

4. Classe de Configuração Swagger (SwaggerConfig)

Crie a classe de configuração para personalizar as informações da API que aparecem no cabeçalho do Swagger UI:

```
package br.com.alloha.rabbitmq.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class SwaggerConfig {

    @Bean
    public OpenAPI openAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("API RabbitMQ - Envio de E-mails")
                .description("API para envio de mensagens de e-mail utilizando
fila RabbitMQ")
            );
    }
}
```

O método `openAPI()` retorna um objeto `OpenAPI` que o SpringDoc usa internamente para gerar a especificação. O `.info()` define o título e descrição da API.

5. Anotação @Tag no Controller

A anotação @Tag agrupa todos os endpoints do controller em uma mesma seção no Swagger UI:

```
import io.swagger.v3.oas.annotations.tags.Tag;

@RestController
@Tag(name = "Mensagens", description = "Endpoints para envio e consulta de mensagens via RabbitMQ")
public class RabbitMQController { ... }
```

- Efeito no Swagger:
- Cria uma seção chamada "Mensagens"
- Todos os endpoints do controller ficam agrupados
- A descrição aparece como texto explicativo

6. Anotação @Operation nos Endpoints

@Operation documenta cada método individualmente, definindo título e descrição:

```
import io.swagger.v3.oas.annotations.Operation;

@Operation(
    summary = "Enviar e-mail para a fila",
    description = "Recebe os dados do e-mail, persiste no banco e envia para a fila RabbitMQ"
)
@PostMapping("/envio-email")
public ResponseEntity<RabbitMQModel> envioEmail(...)
```

summary: Título curto do endpoint (aparece como nome da operação)

description: Descrição detalhada (aparece ao expandir o endpoint)

7. Anotação @ApiResponses e @ApiResponse

@ApiResponses e @ApiResponse documentam todos os códigos HTTP de retorno possíveis:

```
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;

@ApiResponses(value = {
    @ApiResponse(responseCode = "201", description = "Mensagem enviada com
    sucesso",
        content = @Content(schema = @Schema(implementation =
    RabbitMQModel.class))),
    @ApiResponse(responseCode = "400", description = "Dados inválidos
    fornecidos",
        content = @Content(schema = @Schema(implementation =
    ErroResposta.class))),
    @ApiResponse(responseCode = "503", description = "RabbitMQ indisponível",
        content = @Content(schema = @Schema(implementation =
    ErroResposta.class))),
    @ApiResponse(responseCode = "500", description = "Erro interno do servidor",
        content = @Content(schema = @Schema(implementation =
    ErroResposta.class)))
})
@PostMapping("/envio-email")
```

Cada @ApiResponse mapeia:

- responseCode → Código HTTP (201, 400, 503, 500)
- description → Mensagem explicativa
- content → Schema do corpo da resposta (opcional)

8. Anotação @Schema em DTOs e Models

@Schema descreve cada campo dos objetos que aparecem como request/response no Swagger:

```
@Schema(description = "DTO de entrada para envio de e-mail")
public class RabbitMQDTO {

    @Schema(description = "E-mail do destinatário",
            example = "usuario@exemplo.com",
            requiredMode = Schema.RequiredMode.REQUIRED)
    private String email;

    @Schema(description = "Nome do destinatário",
            example = "João Silva",
            requiredMode = Schema.RequiredMode.REQUIRED)
    private String nome;

    @Schema(description = "Conteúdo da mensagem",
            example = "Olá, este é um e-mail de teste.",
            requiredMode = Schema.RequiredMode.REQUIRED)
    private String mensagem;

    @Schema(description = "Observação opcional",
            example = "Envio em lote - campanha promocional")
    private String observacao;
}
```

- Principais atributos do @Schema:
- description → Texto explicativo do campo
- example → Valor de exemplo exibido na documentação
- requiredMode → Se é obrigatório (REQUIRED ou NOT_REQUIRED)
- allowableValues → Valores permitidos (para enums)
- accessMode → READ_ONLY para campos gerados automaticamente (como id)

9. @Content e @Schema em Responses

Para documentar o corpo da resposta de erro, use @Content com @Schema:

```
@ApiResponse(responseCode = "503",
    description = "RabbitMQ indisponível - falha de conexão ou envio",
    content = @Content(
        schema = @Schema(
            implementation = ErroResposta.class
        )
    )
)
```

Isso faz o Swagger mostrar o JSON de exemplo do ErroResposta (com timestamp, status, erro, mensagem, caminho) na documentação do endpoint.

10. Como acessar o Swagger UI

Siga os passos abaixo para visualizar a documentação:

Passo 1: Iniciar os containers Docker

```
cd "C:\Users\joao.velho\Desktop\Projeto RabbitMQ\rabbitmq\rabbitmq"  
docker compose up -d
```

Passo 2: Iniciar a aplicação Spring Boot

```
.\mvnw.cmd spring-boot:run
```

Passo 3: Abrir o navegador

Acesse a URL:

<http://localhost:8080/swagger-ui.html>

Passo 4: Explorar os endpoints

- Expanda a seção "Mensagens"
- Clique em "POST /envio-email" para ver os detalhes
- Clique no botão "Try it out" para testar
- Preencha o JSON no body e clique em "Execute"

11. Como baixar a especificação OpenAPI (JSON)

Para baixar o arquivo JSON da especificação:

Opção 1: Pelo navegador

Acesse e salve a página (Ctrl+S):

```
http://localhost:8080/api-docs
```

Opção 2: Pelo terminal

```
curl http://localhost:8080/api-docs -o swagger-spec.json
```

Esse JSON pode ser importado no Postman, Swagger Editor (editor.swagger.io), ou convertido para HTML estático.

12. Tratamento de Erros com @RestControllerAdvice

Para documentar erros corretamente, criamos um handler global com @RestControllerAdvice:

```
package br.com.alloha.rabbitmq.handler;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MensagemNaoEnviadaException.class)
    public ResponseEntity<ErroResposta> handleMensagemNaoEnviada(
        MensagemNaoEnviadaException ex, WebRequest request) {
        ErroResposta erro = ErroResposta.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.SERVICE_UNAVAILABLE.value())
            .erro("Service Unavailable")
            .mensagem(ex.getMessage())
            .caminho(getCaminho(request))
            .build();
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).body(erro);
    }
}
```

- Exceções tratadas:
- MensagemNaoEnviadaException → 503 (falha ao enviar para fila)
- ConexaoException → 503 (RabbitMQ indisponível)
- AmqpConnectException → 503 (conexão recusada)
- MethodArgumentNotValidException → 400 (validação de campos)
- Exception genérica → 500 (erro interno)

13. Integração ErroResposta no Swagger

O DTO ErroResposta documenta a estrutura padronizada de erro que o Swagger exibirá:

```
@Schema(description = "Estrutura padrão de resposta de erro")
public class ErroResposta {

    @Schema(description = "Data e hora do erro", example = "2026-06-18T12:00:00")
    private LocalDateTime timestamp;

    @Schema(description = "Código HTTP do erro", example = "503")
    private Integer status;

    @Schema(description = "Descrição do erro", example = "Service Unavailable")
    private String erro;

    @Schema(description = "Mensagem detalhada do erro",
              example = "Não foi possível enviar a mensagem para a fila RabbitMQ")
    private String mensagem;

    @Schema(description = "Caminho do endpoint", example = "/envio-email")
    private String caminho;

    @Schema(description = "Erros de validação dos campos")
    private List<ErroValidacao> errosValidacao;
}
```

Exemplo de resposta 503 exibida no Swagger:

```
{
  "timestamp": "2026-06-18T12:00:00",
  "status": 503,
  "erro": "Service Unavailable",
  "mensagem": "Não foi possível conectar ao RabbitMQ.",
  "caminho": "/envio-email"
}
```

14. Como importar no Postman

Existem duas formas de importar a documentação no Postman:

Forma 1: Pelo Swagger/OpenAPI

- Acesse <http://localhost:8080/api-docs>
- Copie todo o JSON
- No Postman: File → Import → Raw text
- Cole o JSON e clique em "Import"
- Pronto! Todos os endpoints criados com exemplos de request/response

Forma 2: Pelo arquivo da collection

- Vá até: Projeto RabbitMQ\rabbitmq\rabbitmq\Postman\
 - Selecione: RabbitMQ-API.postman_collection.json
 - Postman: File → Import → File
 - Selecione o arquivo e clique em "Import"
-
- A collection contém:
 - POST /envio-email → com body de exemplo e respostas 201, 400 e 503
 - GET /mensagens → com resposta 200 e lista de exemplo
 - Swagger UI → link para documentação
 - OpenAPI JSON → link para spec

15. Conclusão

A documentação Swagger/OpenAPI foi totalmente implementada com:

- SpringDoc OpenAPI 2.8.6 integrado ao Spring Boot 4.1.0
- Interface interativa em <http://localhost:8080/swagger-ui.html>
- Especificação JSON em <http://localhost:8080/api-docs>
- Documentação de todos os endpoints com `@Operation` e `@ApiResponse`s
- Modelos e DTOs documentados com `@Schema`
- Tratamento de erros global com `ErroResposta` padronizado
- Documentação de erros (400, 503, 500) integrada ao Swagger
- Collection do Postman completa com exemplos

Com esta documentação, qualquer desenvolvedor pode entender e testar a API sem precisar abrir o código fonte. A combinação do Swagger UI com o Postman permite testar todos os cenários: sucesso, validação e erros.